

Universidad Carlos III de Madrid

Departamento de Informática

Área de Arquitectura y Tecnología de Computadores



Sistemas Distribuidos

Práctica. Servicio de envío de mensajes

Parte 2

Grado de Ingeniería en Informática

Grupo docente de Sistemas Distribuidos

Curso 2025-2026

Índice

1	Objetivo	2
2	Parte 1. Transferencia de ficheros entre usuarios	2
2.1	Envío de un mensaje	2
2.2	Protocolo de envío de mensaje con fichero adjunto cliente-servidor	4
2.3	Envío de un mensaje servidor-cliente	4
2.4	Modificación de la operación de solicitud de usuarios conectados	5
2.5	Solicitud de transferencia de ficheros	6
3	Parte 2. Servicio Web	7
4	Parte 3. RPC	7
5	Normas generales	8
5.1	Calificación de la práctica	9
6	Documentación a entregar	9
6.1	Fichero a entregar	11

1 Objetivo

El objetivo de esta parte de la práctica es completar la funcionalidad del servicio de envío de mensajes, incorporando el envío de ficheros entre usuarios y el empleo de servicios web y RPC.

El esquema final de la aplicación es el que se muestra en la Figura 1.

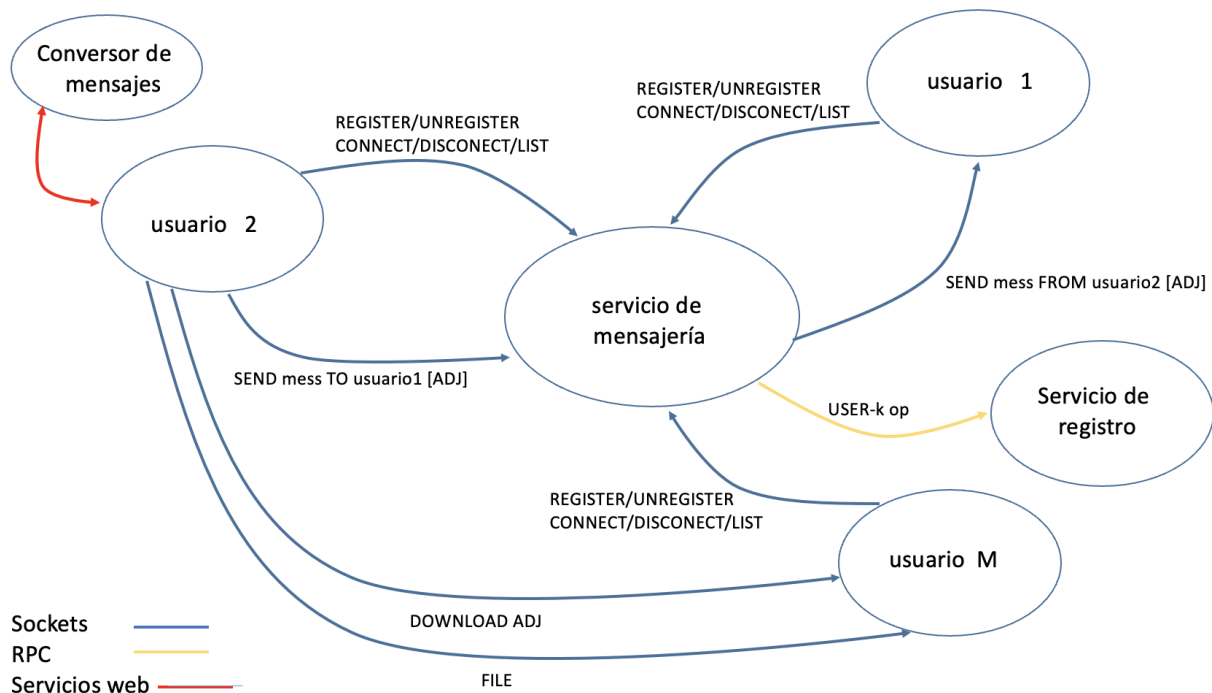


Fig. 1: Interfaz de Usuario

2 Parte 1. Transferencia de ficheros entre usuarios

En esta parte de la práctica se implementará el servicio de transferencia de ficheros entre usuarios. En la transferencia de ficheros no interviene para nada el servidor. Para los nombres de los ficheros se utilizarán siempre nombres con *path* absoluto, como por ejemplo: /tmp/datos.txt.

2.1 Envío de un mensaje

La funcionalidad **SENDATTACH** se usará para establecer conversaciones con el resto de usuarios registrados en el sistema para enviar un mensaje y un fichero.

Para enviar un mensaje a un destinatario incluyendo un fichero se escribirá en la consola:

```
c> SENDATTACH <userName> <message> <fileName>
```

donde:

- `<userName>` indica el nombre del usuario destinatario.
- `<message>` es el mensaje a enviar.
- `<fileName>` indica el nombre del fichero a enviar.

La funcionalidad de esta operación es idéntica a la funcionalidad **SEND** de la parte 1, con la única diferencia que el programa cliente envía también el nombre del fichero.

De esta forma, el servidor asociará a cada mensaje enviado por un usuario un número entero como identificador y llevará siempre el registro de cuál ha sido el último identificador asignado a un mensaje de un usuario. Cuando un usuario se registra por primera vez en el sistema, este identificador se pone a 0, de forma que el primer mensaje que se envía toma como identificador el valor 1, el segundo el valor 2, y así sucesivamente. Cuando se llegue al máximo número de identificadores posibles, el nuevo identificador a asignar volverá a ser el 1, y se procederá de forma similar. El identificador debe almacenarse en una variable de tipo **unsigned int**, cuando se llegue al número máximo representable en una variable de este tipo y se le sume 1, la variable volverá a tomar valor 0 y se continuará el proceso, de forma que el siguiente identificador volverá a ser el 1.

Cuando se envía un mensaje con fichero al servidor, éste devuelve un byte con tres posibles valores (se describen con en la parte 1 de la práctica): un 0 en caso de éxito, un 1 si el usuario no existe y 2 en cualquier otro caso. En caso de éxito (código 0), además devolverá el identificador asociado al mensaje enviado (un número entero) y se mostrará el siguiente mensaje:

```
c> SENDATTACH OK - MESSAGE <id>
```

En caso de que se envíe un mensaje a un usuario no registrado, el servidor indicará el error (código 1) y mostrará en la consola del cliente:

```
c> SENDATTACH FAIL, USER DOES NOT EXIST
```

En caso de que se produzca un error (servidor caído, error de comunicaciones, error por problemas de almacenamiento de mensaje o se devuelva un error de tipo 2) se mostrará:

```
c> SENDATTACH FAIL
```

Una vez que el servidor almacena un mensaje y el nombre de fichero para un usuario y ha respondido con el código correspondiente al usuario remitente, si el usuario está conectado en ese momento le enviará el mensaje junto con el nombre del fichero. En caso de que se haya enviado con éxito, el servidor enviará al remitente del mensaje la confirmación de que el mensaje con el identificador asignado se ha enviado al usuario correctamente, junto con el nombre del fichero. Cada vez que un cliente remitente de un mensaje recibe del servidor un mensaje de entrega de mensaje a otro proceso junto con un nombre de fichero mostrará:

```
c> SENDATTACH MESSAGE <id> <fileName> OK
```

Indicando que el mensaje con identificador `<id>` se ha entregado correctamente, y que el nombre de fichero asociado es `fileName`.

En caso de que el usuario no esté conectado, el servidor almacenará el mensaje y el nombre de fichero. Posteriormente cuando el cliente destinatario se conecte el servidor se encargará de enviarle todos los mensajes pendientes (uno a uno), junto con los nombres de fichero. Cada vez

que se envía con éxito un mensaje a un usuario, se notifica el remitente del mensaje, el cual mostrará por pantalla:

```
c> SENDATTACH MESSAGE <id> <fileName> OK
```

Siempre que el servidor envía con éxito un mensaje a un usuario, descarta el mensaje eliminándolo del servidor. Es decir, el servidor solo almacena los mensajes pendientes de entrega, cada vez que se entrega con éxito un mensaje se borra del servidor.

2.2 Protocolo de envío de mensaje con fichero adjunto cliente-servidor

Cuando el cliente quiere enviarle a otro usuario un mensaje con fichero adjunto realizará las siguientes acciones:

1. Se conecta al servidor, de acuerdo a la IP y puerto pasado en la línea de mandatos al programa.
2. Se envía la cadena "SENDATTACH" indicando la operación.
3. Se envía una cadena con el nombre que identifica al usuario que envía el mensaje.
4. Se envía una cadena con el nombre que identifica al usuario destinatario del mensaje.
5. Se envía una cadena en la que se codifica el mensaje a enviar (como mucho 256 caracteres incluido el código '0', es decir la cadena tendrá como mucho una longitud de 255 caracteres).
6. Se envía una cadena con el nombre del fichero adjunto. Se considerará que el nombre del fichero tiene como mucho una longitud de 255 caracteres.
7. Recibe del servidor un byte que codifica el resultado de la operación: 0 en caso de éxito. En este caso recibirá a continuación una cadena de caracteres que codificará el identificador numérico asignado al mensaje, "132" para el mensaje con número 132. Si no se ha realizado la operación con éxito se recibe 1 si el usuario no existe y 2 en cualquier otro caso. En estos dos casos no se recibirá ningún identificador.
8. Cierra la conexión.

2.3 Envío de un mensaje servidor-cliente

Cuando el servidor quiere enviarle a un usuario registrado y conectado un mensaje, con fichero adjunto, de otro usuario realizará las siguientes acciones (por cada mensaje a enviar):

1. Se conecta al thread de escucha del cliente (de acuerdo a la IP y puerto almacenado para ese cliente).
2. Se envía la cadena "SEND_MESSAGE_ATTACH" indicando la operación.
3. Se envía una cadena con el nombre que identifica al usuario que envía el mensaje.
4. Se envía una cadena codificando en ella el identificador asociado al mensaje.

5. Se envía una cadena con el mensaje (todos los mensajes tendrán como mucho 256 bytes incluido el código '0', este tamaño lo controlará el cliente).
6. Se envía una cadena con el nombre del fichero adjunto.
7. Cierra la conexión.

Si se produce algún error durante esta operación, el mensaje se considerará no entregado y se seguirá almacenando en el servidor como pendiente de entrega, hasta que se pueda entregar. Si se produce algún error durante la conexión a un cliente, el servidor asumirá que el cliente se ha desconectado y lo marcará como desconectado.

Una vez enviado el mensaje, el servidor tiene que notificar al usuario que lo envió (remitente) sobre su recepción. Para ello el servidor realiza las siguientes acciones:

1. Se conecta al thread de escucha del cliente remitente del mensaje (de acuerdo a la IP y puerto almacenado para ese cliente).
2. Se envía la cadena "SEND_MESS_ATTACH_ACK" indicando la operación.
3. Se envía una cadena codificando en ella el identificador asociado al mensaje que se ha entregado.
4. Se envía una cadena codificando en ella el nombre del fichero adjunto.
5. Cierra la conexión.

En caso de que el usuario remitente no estuviera conectado, se descartará este mensaje y no se realizarán las acciones anteriores.

2.4 Modificación de la operación de solicitud de usuarios conectados

Para que un usuario pueda solicitar la transferencia de un fichero a otro, es necesario que el primero conozca la dirección IP y el puerto del segundo. Para ello, se modificará la funcionalidad de solicitud de usuarios conectados de forma que el servidor devuelva por cada usuario conectado, la IP y el puerto de escucha del thread creado en la conexión.

De esta forma, cuando un cliente quiere saber los usuarios conectados en el servicio de mensajería se realizan las siguientes operaciones:

1. Se conecta al servidor, de acuerdo a la IP y puerto pasado en la línea de mandatos al programa.
2. Se envía la cadena "USERS" indicando la operación.
3. Se envía una cadena con el nombre que identifica al usuario que envía el mensaje.
4. Recibe del servidor un byte que codifica el resultado de la operación: 0 en caso de éxito, 1 si el usuario no está conectado, 2 en cualquier otro caso.
5. En caso de éxito (valor devuelto 0), el cliente recibirá del servidor una cadena de caracteres que codifica la cantidad de clientes conectados al servidor. Así, para 18 clientes conectados recibirá la cadena de texto "18".

6. Recibe del servidor tantas cadenas como clientes haya conectados. Una cadena de caracteres por usuario. Cada cadena incluirá la IP y el puerto y estará codificada de la siguiente forma: `usuario :: IP :: puerto`.

7. Cierra la conexión.

La información recibida con los usuarios conectados se almacenará en el cliente Python en una estructura de datos que permitirá conocer para cada usuario conectado, la IP y el puerto asociado al thread creado en la conexión.

2.5 Solicitud de transferencia de ficheros

Cuando un cliente recibe un mensaje de servidor con fichero adjunto, el thread que recibe la petición muestra por pantalla:

```
c> MESSAGE <id> FROM <userName>
    <message>
    END
    FILE <fileName>
```

Donde `<userName>` indica el nombre del usuario y `<fileName>` el nombre del fichero adjunto.

Para poder recuperar el contenido del fichero con nombre `<fileName>` ha de modificarse el código del cliente escrito en Python añadiendo una nueva funcionalidad:

```
c> GETFILE <userName> <fileName> <localFileName>
```

donde:

- `<userName>` indica el nombre del usuario.
- `<fileName>` indica el nombre del fichero remoto que hay que transferir.
- `<localFileName>` indica el nombre del fichero local en el que se ha de copiar el contenido del fichero remoto.

Para poder llevar a cabo la operación, el código del cliente buscará en la estructura de datos creada en la sección anterior, la dirección IP y el puerto asociado al thread de escucha del usuario que posee el fichero. En caso de no encontrarse en esa estructura de datos, se volverá a realizar de forma interna una solicitud de petición de usuarios conectados para refrescar la información.

En caso de que el usuario no se encuentre en la estructura anterior por estar desconectado, se mostrará en la interfaz del cliente el siguiente mensaje:

```
c> FILE TRANSFER FAILED, user not connected.
```

Si se dispone de la IP y del puerto se realizarán las siguientes acciones:

1. Se conecta al thread de escucha del cliente (de acuerdo a la IP y puerto almacenado para ese cliente).

2. Se envía la cadena “GET_FILE” indicando la operación.
3. Se envía una cadena con el nombre que identifica al usuario que envía la operación.
4. Se envía una cadena codificando en ella el nombre del fichero a transferir.
5. Se recibe del cliente el contenido del fichero y se almacena de forma local en `<localFileName>`.
6. Cierra la conexión.

El proceso para transferir el contenido del fichero entre el usuario remoto y el fichero local (acción 5 anterior) es totalmente libre y se puede implementar como se desee, teniendo en cuenta que el contenido ha de transferirse obligatoriamente a través de sockets.

3 Parte 2. Servicio Web

Para el servicio web, se desarrollará y desplegará un servicio web desarrollado en Python siguiendo el material presentado en la asignatura.

Este servicio web se encargará de normalizar los mensajes que se envían los usuarios. Cada vez que un usuario redacta un mensaje, se lo envía a este conversor de mensajes para que elimine del mensaje los espacios en blanco repetidos. El objetivo es que en los mensajes, las diferentes palabras estén separadas solo por un espacio en blanco.

El servicio se desplegará, por simplicidad, en la máquina local donde ejecuta el cliente desarrollado en Python.

4 Parte 3. RPC

Esta parte de la práctica pretende ampliar la aplicación desarrollada en la Parte 1 para añadir un servicio, basado en RPC, que se encargue de imprimir por pantalla las operaciones que realizan los usuarios del sistema.

Cada vez que el servidor reciba una operación de un usuario enviará al servidor RPC el nombre del usuario que realiza la operación y la operación que realiza. Toda esta información se enviará como cadenas de caracteres.

De esta forma, cada vez que el servidor reciba una petición remota imprimirá la siguiente información:

Nombre_usuario OPERACION

Donde OPERACION puede tomar los siguientes valores:

- REGISTER
- UNREGISTER
- CONNECT
- DISCONNECT
- USERS

- SEND
- SENDATTACH

En el caso de SENDATTACH también se imprimirá el nombre del fichero enviar, como se puede ver en el siguiente ejemplo:

```
Nombre_usuario      SENDATTACH  /tmp/file.txt
```

Se deja total libertad para definir la interfaz (fichero .x) que se considere más adecuada. En todo caso, es necesario que se justifique la interfaz definida.

Tenga en cuenta que solo se envía el nombre de la operación y el nombre del fichero para SENDATTACH. No hay que enviar el texto de los mensajes que se envían los usuarios.

Para el desarrollo de esta parte se utilizará el lenguaje de programación C y el modelo ONC-RPC.

El proceso servidor de la parte 1 es cliente del servicio RPC implementado en esta parte, mientras que el servidor ONC-RPC desarrollado ofrece el servicio. El servidor de la parte 1 accederá a esta variable de entorno para poder localizar el servicio.

Para poder ejecutar correctamente el servidor de la parte 1, este necesitará conocer la dirección IP o nombre del computador donde ejecuta el servidor RPC. Para ello se definirá una variable de entorno (denominado LOG_RPC_IP) de forma similar a como se hizo en los ejercicios evaluables 2 y 3.

Para poder probar toda la funcionalidad será necesario:

- Ejecutar el servicio web (en la máquina local del cliente).
- Ejecutar el servidor RPC.
- Ejecutar el servidor de la parte 1 con la variable de entorno LOG_RPC_IP correctamente definida.
- Ejecutar los clientes que considere necesario para probar el proyecto. En caso de que se pruebe con varios clientes en varias máquinas, cada máquina cliente tendrá su propio servicio web desplegado.

5 Normas generales

Para el desarrollo de las dos partes que constituyen la práctica han de seguirse las siguientes normas:

1. Las prácticas que no compilen o no se ajusten a la funcionalidad y requisitos planteados, **obtendrán una calificación de 0.**
2. Las prácticas que tengan *warnings* **serán penalizadas.**
3. Un programa no comentado, **obtendrá una calificación de 0.**
4. La entrega de la práctica se realizará a través de los entregadores habilitados. **No se permite la entrega a través de correo electrónico.**

5. Se prestará especial atención a detectar funcionalidades copiadas entre dos prácticas. En caso de detectar copia, ambos grupos **perderán la evaluación continua**.
6. **Toda la práctica tendrá que desarrollarse y funcionar correctamente en las aulas de laboratorio utilizadas en la asignatura.**
7. El sistema debe funcionar con clientes y servidores ejecutando en contenedores con direcciones IP distintas.
8. La memoria debe tener una **longitud máxima de 15 páginas**. No se incluirán capturas de pantalla en la sección de pruebas.

5.1 Calificación de la práctica

Sólo debe hacerse una entrega que podrá contener la funcionalidad completa de todas las partes o de solo la parte 1. La práctica se calificará de la siguiente forma:

- La parte 1 de la práctica se puntuará sobre 6 puntos.
- El envío de ficheros adjuntos (en la parte 2) se puntuará sobre 2 puntos.
- El servicio web (parte 2) se puntuará sobre 1 punto.
- El servicio RPC (parte 2) se puntuará sobre 1 punto.

De esta forma si solo se entrega la parte 1, como máximo se obtendrán 6 puntos. La entrega de las partes 1 y 2 permitiría obtener hasta 10 puntos. En todo caso, será obligatorio entregar la parte 1. Es opcional entregar la parte 2. Se puede entregar solo la parte 1 o la parte 1 y 2. Dentro de la parte 2 se pueden entregar de forma opcional **cualquiera** de estas partes:

- El envío de ficheros adjuntos.
- El servicio web.
- El servicio RPC.

6 Documentación a entregar

La práctica se desarrollará en grupos de dos alumnos como máximo. La práctica sólo deberá ser entregada por un único integrante del grupo de prácticas en su grupo docente. No se debe entregar la misma práctica de forma repetida por todos los integrantes del grupo.

El plazo de entrega de toda la práctica en su conjunto finaliza el **10 de mayo de 2026**.

La entrega se realizará mediante Aula Global, a través de un entregador que se habilitará a tal efecto.

Se debe entregar un fichero comprimido en formato zip con el nombre **ssdd_proyecto_A_B.zip** donde A y B son los NIA de los integrantes que realizan la entrega.

El fichero en formato zip debe contener:

- `autores.txt`, con los nombres y NIA de los integrantes del grupo.
- `memoria.pdf`
- `client.py`
- `server.c` y/o todos los ficheros fuentes que necesite el servidor para su compilación.
- Fichero `Makefile` utilizado para compilar todos los archivos `.c`.
- Ficheros Python necesarios para el desarrollo del servicio web.
- Fichero con la interfaz (`.x`) del servidor RPC y todos los ficheros necesarios para la compilación y ejecución.
- Fichero de texto de nombre `README` con instrucciones detalladas para la compilación y despliegue de todos los procesos involucrados en la aplicación.
- Cualquier otro fichero fuente que se considere necesario para la compilación o evaluación de la práctica.

Los ficheros entregados deben incluir la funcionalidad de todas las partes que se hayan completado.

Deben incluirse todos los archivos fuente necesarios para la compilación y un fichero de texto con nombre `README`, que incluirá instrucciones detalladas para la compilación y despliegue de todos los procesos involucrados en la aplicación.

La memoria de la práctica debe comentar los aspectos del desarrollo de la misma que considere más relevantes. Del mismo modo, puede exponer los comentarios personales que considere oportunos. Se deberá entregar un documento en formato PDF.

No descuide la calidad de la memoria de su práctica. Aprobar la memoria es imprescindible para aprobar la práctica, tanto como el correcto funcionamiento de la misma. **Si al evaluarse la memoria de su práctica, se considera que no alcanza el mínimo admisible, su práctica estará suspensa.**

La memoria tendrá que contener al menos los siguientes apartados:

- **Portada** donde figuren los autores (incluyendo nombre completo, NIA y dirección de correo electrónico).
- **Índice de contenidos.**
- **Descripción del código** detallando las principales funciones implementadas. No incluir código fuente de la práctica en este apartado.
- **Descripción de la forma de compilar y obtener el ejecutable de todos los procesos involucrados.** Además, se debe describir la forma de ejecutarlos.
- **Batería de pruebas** utilizadas y resultados obtenidos. Se dará mayor puntuación a pruebas avanzadas, casos extremos, y en general a aquellas pruebas que garanticen el correcto funcionamiento de la práctica en todos los casos.

Hay que tener en cuenta:

- Que el programa compile correctamente y sin *warnings* a ser posible.
- Evite pruebas duplicadas que evalúan los mismo flujos de programa. La puntuación de este apartado no se mide en función del número de pruebas, sino del grado de cobertura de las mismas. Es mejor pocas pruebas que evalúan diferentes casos, a muchas que evalúan siempre el mismo caso.
- **Conclusiones**, problemas encontrados, cómo se han solucionado, y opiniones personales. Se puntuarán también los siguientes aspectos relativos a la **presentación**:
 - La memoria debe tener números de página en todas las páginas (menos en la portada).
 - El texto de la memoria debe estar justificado.

6.1 Fichero a entregar

Para crear el fichero a entregar se deben seguir los siguientes pasos:

- Se crea el directorio para preparar los materiales a entregar y se comprueba que se encuentra en el directorio de la entrega:

```
$ cd
$ mkdir ssdd_proyecto_AAAAAAAAAA_BBBBBBBBBB
$ cd ssdd_proyecto_AAAAAAAAAA_BBBBBBBBBB
```

- Después se procederá a copiar todos los ficheros con los programas desarrollados al directorio de la entrega y se procede a generar el fichero zip a ser entregado:

```
$ cd ..
$ ls
... ssdd_proyecto_AAAAAAAAAA_BBBBBBBBBB ...
$ zip -r ssdd_proyecto_AAAAAAAAAA_BBBBBBBBBB.zip ssdd_p2_AAAAAAAAAA_BBBBBBBBBB/
```

Solo se hará una única entrega para todas las partes que compone la práctica. La fecha tope de entrega es el **10 de mayo de 2026** a las 23:55.